

EXPERIMENT 3

Aim

To create a Cryptocurrency using Python and perform mining in the Blockchain created, and to implement a public/private Blockchain to demonstrate decentralized consensus in a real-world simulated network.

Theory

1. Blockchain Overview

A blockchain is a distributed and decentralized ledger made up of a series of linked blocks. Each block stores transaction data, a timestamp, the hash of the previous block, and its own unique hash (a digital fingerprint generated from its contents). Because each block's hash depends on the previous block's hash, altering any block would require recalculating every block after it — this is what makes the blockchain tamper-resistant and immutable.

2. Mining

Mining is the process by which new blocks are added to the chain. It involves:

- Collecting pending transactions from the transaction pool into a candidate block.
- Solving a computational puzzle (Proof-of-Work) to find a valid proof/nonce whose resulting hash satisfies a difficulty condition (e.g., starts with a fixed number of zeros).
- Appending the new block to the chain and broadcasting it to peers.
- Rewarding the miner with a small amount of cryptocurrency for their computational effort.

3. Multi-Node (P2P) Blockchain Network

The lab simulates three independent nodes running on ports 5001, 5002, and 5003. Each node runs its own Flask server, maintains its own local copy of the blockchain, and can connect to peer nodes to share, validate, and synchronize block data — mimicking a real peer-to-peer cryptocurrency network.

4. Consensus Mechanism — Longest Chain Rule

Since each node can mine independently, different nodes may temporarily hold

different versions of the chain. Consensus is achieved through the Longest Chain Rule: when nodes disagree, the longest *valid* chain among all peers is accepted as the true chain, ensuring the entire network eventually agrees on a single transaction history.

5. Transactions and Mining Reward

Each transaction records a sender, receiver, and amount, and is held in a pending transaction pool until it is mined into a block. When a node mines a block, all pending transactions are packed into it, a mining-reward transaction is automatically added crediting the miner, and the pool is cleared for the next round.

6. Chain Replacement

When the `/replace_chain` endpoint is called, a node requests the current chain from each of its connected peers. If a peer's chain is both longer and valid (verified via `is_chain_valid`), the node replaces its own chain with it — keeping the ledger consistent network-wide.

Tools & Libraries Used

- Python 3.x
- Flask — lightweight web framework used to expose the blockchain's API endpoints
- Requests — used for HTTP communication between nodes during chain replacement
- Postman — used to test and invoke the API endpoints
- Standard libraries: `datetime` (timestamps), `hashlib` (SHA-256 hashing), `uuid4` (unique node/miner addresses), `urlparse` (parsing peer node addresses), `jsonify/request` (Flask request-response handling)

CODE:

```
"""
HadCoin - A simple Proof-of-Work Cryptocurrency Blockchain Node
Node Port: 5001
```

```
Run with: python hadcoin_node_5001.py
```

```
Requires: pip install Flask requests
```

```
"""
```

```
import datetime
import hashlib
import json
from uuid import uuid4
from urllib.parse import urlparse

import requests
from flask import Flask, jsonify, request
```

```
#
```

```
-----
```

```
-
```

```
# Part 1 - Building the Blockchain
```

```
#
```

```
-----
```

```
-
```

```
class Blockchain:
```

```
    def __init__(self):
        self.chain = []
        self.transactions = []
        # Genesis block
        self.create_block(proof=1, previous_hash='0')
        self.nodes = set()
```

```
    def create_block(self, proof, previous_hash):
        block = {
            'index': len(self.chain) + 1,
```

```

        'timestamp': str(datetime.datetime.now()),
        'proof': proof,
        'previous_hash': previous_hash,
        'transactions': self.transactions,
    }
    # Transactions are cleared once they are packed into a block
    self.transactions = []
    self.chain.append(block)
    return block

def get_previous_block(self):
    return self.chain[-1]

def proof_of_work(self, previous_proof):
    new_proof = 1
    check_proof = False
    while not check_proof:
        hash_operation = hashlib.sha256(
            str(new_proof ** 2 - previous_proof ** 2).encode()
        ).hexdigest()
        if hash_operation[:4] == '0000':
            check_proof = True
        else:
            new_proof += 1
    return new_proof

def hash(self, block):
    encoded_block = json.dumps(block, sort_keys=True).encode()
    return hashlib.sha256(encoded_block).hexdigest()

def is_chain_valid(self, chain):
    previous_block = chain[0]
    block_index = 1
    while block_index < len(chain):
        block = chain[block_index]
        if block['previous_hash'] != self.hash(previous_block):
            return False
        previous_proof = previous_block['proof']
        proof = block['proof']
        hash_operation = hashlib.sha256(

```

```

        str(proof ** 2 - previous_proof ** 2).encode()
    ).hexdigest()
    if hash_operation[:4] != '0000':
        return False
    previous_block = block
    block_index += 1
    return True

def add_transaction(self, sender, receiver, amount):
    self.transactions.append({
        'sender': sender,
        'receiver': receiver,
        'amount': amount,
    })
    previous_block = self.get_previous_block()
    return previous_block['index'] + 1

# Part 2 - Decentralizing the Blockchain
-----

def add_node(self, address):
    parsed_url = urlparse(address)
    self.nodes.add(parsed_url.netloc)

def replace_chain(self):
    network = self.nodes
    longest_chain = None
    max_length = len(self.chain)
    for node in network:
        response = requests.get(f'http://{node}/get_chain')
        if response.status_code == 200:
            length = response.json()['length']
            chain = response.json()['chain']
            if length > max_length and self.is_chain_valid(chain):
                max_length = length
                longest_chain = chain
    if longest_chain:
        self.chain = longest_chain
        return True
    return False

```

```

#
-----
-
# Part 3 - Mining the Blockchain / Running the App as an API
#
-----
-
app = Flask(__name__)

# Address of this node (used as the "sender" for mining rewards)
node_address = str(uuid4()).replace('-', '')

# Create the Blockchain
blockchain = Blockchain()

@app.route('/', methods=['GET'])
def home():
    return jsonify({'message': 'HadCoin node is running', 'node_address':
node_address}), 200

@app.route('/mine_block', methods=['GET'])
def mine_block():
    previous_block = blockchain.get_previous_block()
    previous_proof = previous_block['proof']
    proof = blockchain.proof_of_work(previous_proof)
    previous_hash = blockchain.hash(previous_block)
    # Reward the miner with 1 HadCoin
    blockchain.add_transaction(sender=node_address, receiver='Richard',
amount=1)
    block = blockchain.create_block(proof, previous_hash)
    response = {
        'message': 'Congratulations, you just mined a block!',
        'index': block['index'],
        'timestamp': block['timestamp'],
        'proof': block['proof'],
        'previous_hash': block['previous_hash'],
        'transactions': block['transactions'],

```

```

    }
    return jsonify(response), 200

@app.route('/get_chain', methods=['GET'])
def get_chain():
    response = {'chain': blockchain.chain, 'length':
len(blockchain.chain)}
    return jsonify(response), 200

@app.route('/is_valid', methods=['GET'])
def is_valid():
    is_valid_chain = blockchain.is_chain_valid(blockchain.chain)
    if is_valid_chain:
        response = {'message': 'All good. The blockchain is valid.'}
    else:
        response = {'message': 'Houston, we have a problem. The chain is
not valid.'}
    return jsonify(response), 200

@app.route('/add_transaction', methods=['POST'])
def add_transaction():
    json_data = request.get_json()
    transaction_keys = ['sender', 'receiver', 'amount']
    if not all(key in json_data for key in transaction_keys):
        return 'Some elements of the transaction are missing', 400
    index = blockchain.add_transaction(
        json_data['sender'], json_data['receiver'], json_data['amount']
    )
    response = {'message': f'This transaction will be added to Block
{index}'}
    return jsonify(response), 201

# Part 4 - Decentralizing the Blockchain
-----

@app.route('/connect_node', methods=['POST'])
def connect_node():

```

```

    json_data = request.get_json()
    nodes = json_data.get('nodes')
    if nodes is None:
        return "No nodes provided. Send a list under the 'nodes' key, e.g.
" \
        '{"nodes": ["http://127.0.0.1:5002",
"http://127.0.0.1:5003"]}', 400
    for node in nodes:
        blockchain.add_node(node)
    response = {
        'message': 'All the nodes are now connected. The HadCoin
Blockchain now contains the following nodes:',
        'total_nodes': list(blockchain.nodes),
    }
    return jsonify(response), 201

@app.route('/replace_chain', methods=['GET'])
def replace_chain():
    is_chain_replaced = blockchain.replace_chain()
    if is_chain_replaced:
        response = {
            'message': 'The nodes had different chains so the chain was
replaced by the longest one.',
            'new_chain': blockchain.chain,
        }
    else:
        response = {
            'message': 'All good. The chain is the largest one.',
            'actual_chain': blockchain.chain,
        }
    return jsonify(response), 200

#
-----
-
# Running the app
#
-----

```



```

if __name__ == '__main__':
    # host='0.0.0.0' so the node is reachable when deployed/hosted, not
    just on localhost
    app.run(host='0.0.0.0', port=5001, debug=True)

```

Output:

HadCoin Blockchain (3 Nodes) > Node 1 (5001) > Add Transaction

POST {{node1_url}}/add_transaction

Send

Docs Params Authorization Headers 9 Body Scripts Settings Cookies

raw Text

```

1 {
2   "sender": "rutuja",
3   "receiver": "isha",
4   "amount": 100
5 }

```

Body Cookies Headers 5 Test Results 201 Created 2 ms 233 B

Save Response

JSON Preview Visualize

```

1 {
2   "message": "This transaction will be added to Block 2"
3 }
4

```

HadCoin Blockchain (3 Nodes) > Node 1 (5001) > Mine Block

GET {{node1_url}}/mine_block

Send

Docs Params Authorization Headers 6 Body Scripts Settings Cookies

Body Cookies Headers 5 Test Results 200 OK 51 ms 528 B

Save Response

JSON Preview Visualize

```

1 {
2   "index": 3,
3   "message": "Congratulations, you just mined a block!",
4   "previous_hash": "8e2e4ee1cd6f0472244acdf7b3ee0982b55e4f9754523260d4bc1d71cf921c7",
5   "proof": 45293,
6   "timestamp": "2026-09-09 13:28:51.441786",
7   "transactions": [
8     {
9       "amount": 1,
10      "receiver": "Richard",
11      "sender": "788c00b4d6234c288bba3698e1d36aa1"
12    }
13  ]
14 }
15

```

HadCoin Blockchain (3 Nodes) > Node 1 (5001) > Get Chain

GET `{{node1_url}}/get_chain` Send

Docs Params Authorization Headers 6 Body Scripts Settings Cookies

Query Params

Key	Value	Description	Bulk Edit	...
Key	Value	Description		

Body Cookies Headers 5 Test Results 200 OK • 3 ms • 1.14 KB Save Response

{ } JSON Preview Visualize

```
30      "previous_hash":
31          "8e2e4ee1cd6f0472244acdff7b3ee0982b55e4f9754523260d4bc1d71cf921c7",
32      "proof": 45293,
33      "timestamp": "2026-09-09 13:28:51.441786",
34      "transactions": [
35          {
36              "amount": 1,
37              "receiver": "Richard",
38              "sender": "788c00b4d6234c288bba3698e1d36aa1"
39          }
40      ],
41      "length": 3
42  }
43  }
```

HadCoin Blockchain (3 Nodes) > Node 2 (5002) > Get Chain

GET `{{node2_url}}/get_chain` Send

Docs Params Authorization Headers 6 Body Scripts Settings Cookies

Query Params

Key	Value	Description	Bulk Edit	...
Key	Value	Description		

Body Cookies Headers 5 Test Results 200 OK • 3 ms • 813 B Save Response

{ } JSON Preview Visualize

```
16      {
17          "amount": 100,
18          "receiver": "isha",
19          "sender": "rutuja"
20      },
21      {
22          "amount": 1,
23          "receiver": "Richard",
24          "sender": "788c00b4d6234c288bba3698e1d36aa1"
25      }
26  ],
27  "length": 2
28  }
```

HadCoin Blockchain (3 Nodes) > Node 2 (5002) > Replace Chain

SaveShare

GET{{node2_url}}/replace_chainSend

DocsParamsAuthorizationHeaders 6BodyScriptsSettingsCookies

Query Params

Key	Value	Description	Bulk Edit	...
Key	Value	Description		

BodyCookiesHeaders 5Test Results200 OK12 ms1.22 KBSave Response

JSONPreviewVisualize

```
1 {
2   "message": "The nodes had different chains so the chain was replaced by the longest one.",
3   "new_chain": [
4     {
5       "index": 1,
6       "previous_hash": "0",
7       "proof": 1,
8       "timestamp": "2026-09-09 13:22:36.581105",
9       "transactions": []
10    },
11    {
12      "index": 2,
13      "previous_hash":
14        "43523be0f88d7631c868ccc6f00a2a09808626e2b60de5552e37b16bc0f31bc9",
15      "proof": 533,
16      "timestamp": "2026-09-09 13:23:34.483863"
```

No anomalies detected!

HadCoin Blockchain (3 Nodes) > Node 2 (5002) > Get Chain

SaveShare

GET{{node2_url}}/get_chainSend

DocsParamsAuthorizationHeaders 6BodyScriptsSettingsCookies

Query Params

Key	Value	Description	Bulk Edit	...
Key	Value	Description		

HadCoin Blockchain (3 Nodes) > Node 2 (5002) > Get Chain

SaveShare

GET{{node2_url}}/get_chainSend

DocsParamsAuthorizationHeaders 6BodyScriptsSettingsCookies

Query Params

Key	Value	Description	Bulk Edit	...
Key	Value	Description		

BodyCookiesHeaders 5Test Results200 OK2 ms1.14 KBSave Response

JSONPreviewVisualize

```
30   "previous_hash":
31     "8e2e4ee1cd6f0472244acdf7b3ee0982b55e4f9754523260d4bc1d71cf921c7",
32   "proof": 45293,
33   "timestamp": "2026-09-09 13:28:51.441786",
34   "transactions": [
35     {
36       "amount": 1,
37       "receiver": "Richard",
38       "sender": "788c00b4d6234c288bba3698e1d36aa1"
39     }
40   ],
41   "length": 3
42 }
43 }
44 }
```

Conclusion

We successfully developed a cryptocurrency using Python and implemented mining within a simulated three-node peer-to-peer blockchain network. Each node maintained its own independent ledger and synchronized with its peers using the Longest Chain Rule to preserve consistency across the network. Mining was carried out through a Proof-of-Work algorithm, transactions were securely added and cleared from the pool once mined, and miners were automatically rewarded. Through this experiment, we gained practical understanding of key blockchain concepts — decentralized consensus, transaction validation, chain integrity via hashing, and network synchronization across multiple nodes.